



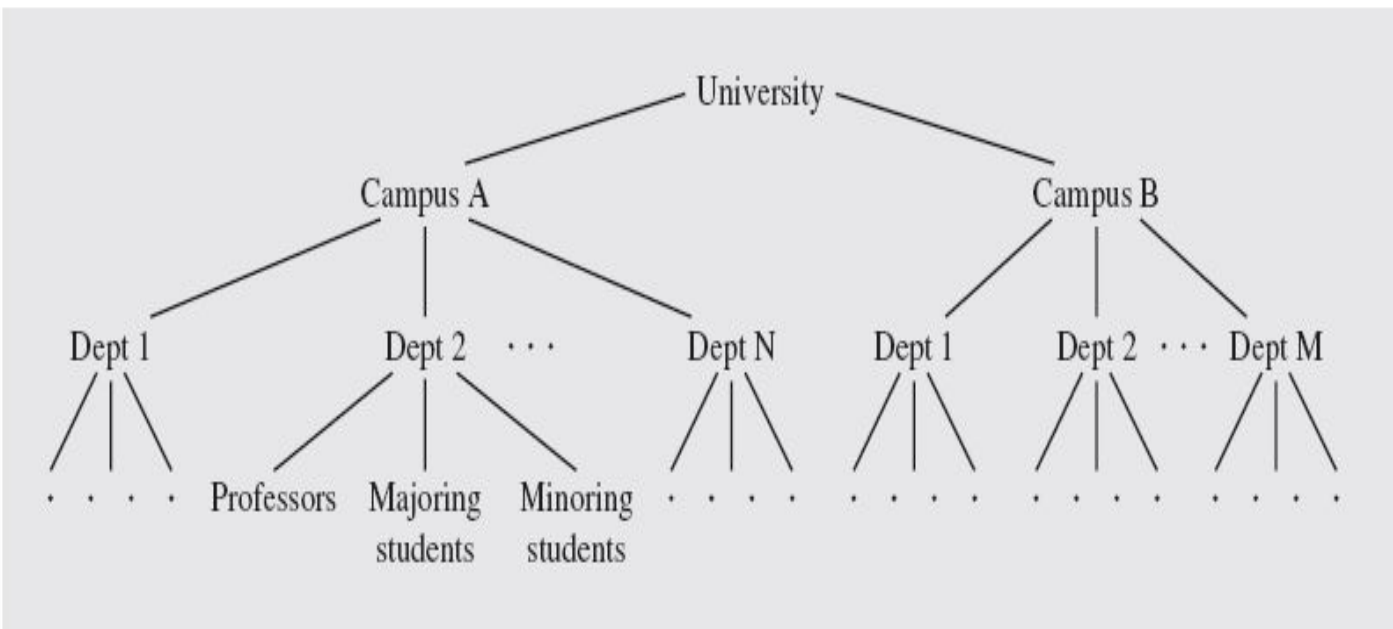
Tree

# Tree

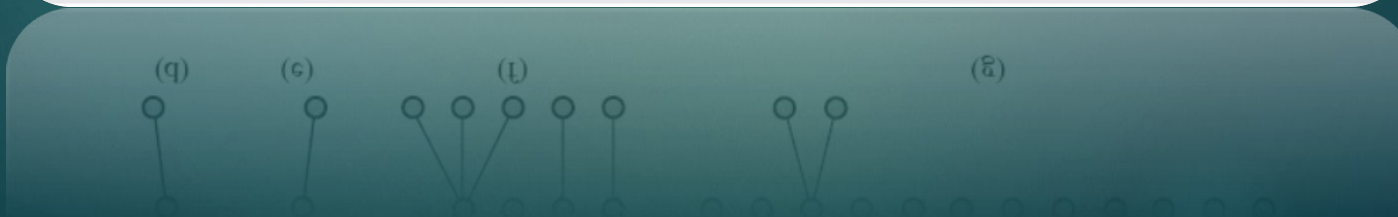
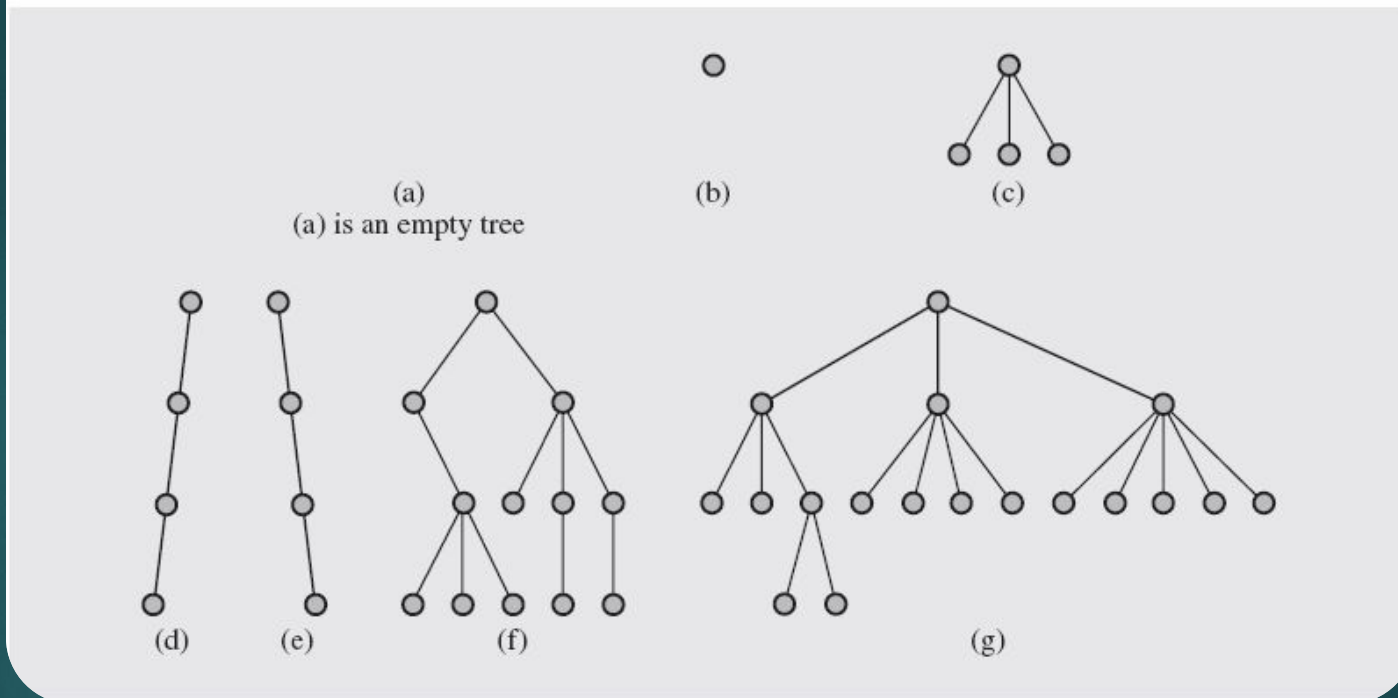
- ▶ Link List , stack , queue etc
  - ▶ linear structures
  - ▶ Difficult to use hierarchical representation of objects.
  - ▶ they are limited to only one dimension.
- ▶ Unlike natural trees, these trees are depicted upside down with the *root at the top and the leaf (terminal nodes) at the bottom.*



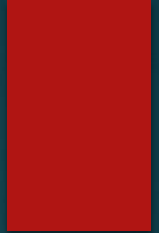
**FIGURE 6.2** Hierarchical structure of a university shown as a tree.



**FIGURE 6.1** Examples of trees.

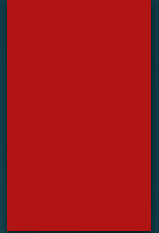


# Tree



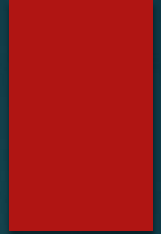
- ▶ Root
  - ▶ has no parent
  - ▶ can have only child nodes.
- ▶ Leaf
  - ▶ have no children, or rather, their children are empty structures.
- ▶ Path
  - ▶ Each node has to be reachable from the root through a unique sequence of arcs(edges), called a *path*

# Tree



- ▶ Length
  - ▶ The number of edges in a path is called the *length of the path*.
- ▶ The empty tree
  - ▶ Do not have any node.
- ▶ Single node is a tree of height 0 or level 0.
  - ▶ This is the only case in which a node is both the root and a leaf

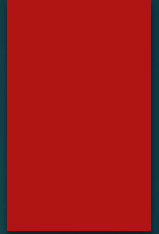
# Tree



## ► Level/Depth

- ❑ The level of a node in a binary tree is defined as follows:
- ❑ Root has level 0,
- ❑ Level of any other node is one more than the level its parent (father).
- ❑ The height of a tree (sometimes called the "depth" of a tree) is a measure of how many levels there are in the tree.
- ❑ It is defined as the length of the longest path from the root node to a leaf node. In other words, it's the maximum number of edges in any path from the root to a leaf.

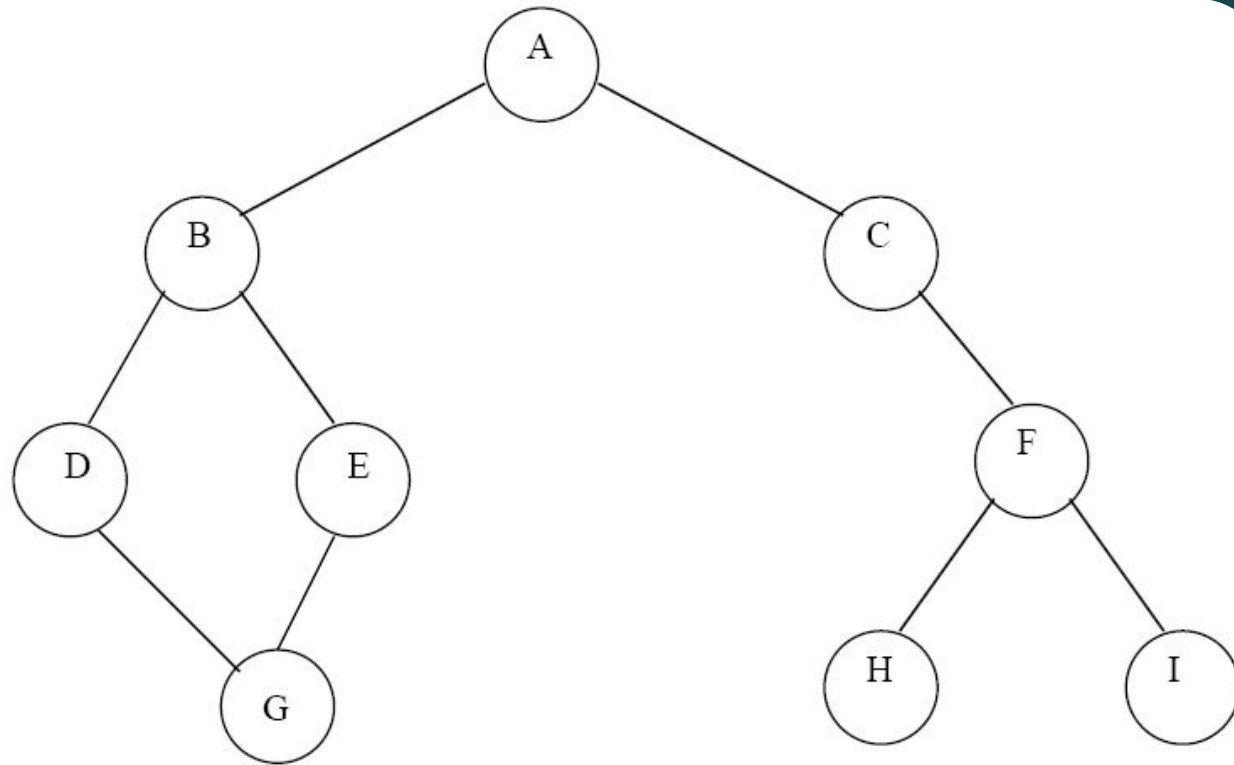
# Binary Trees



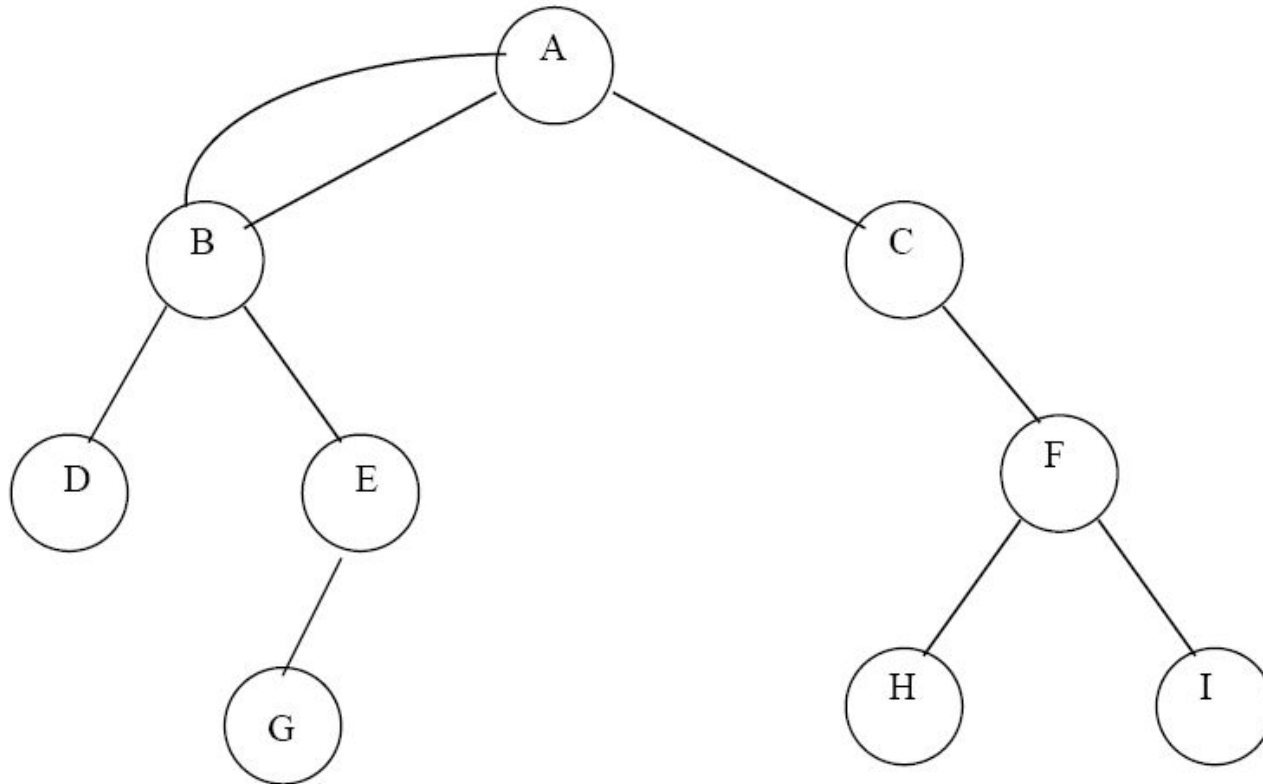
- ▶ A binary tree is a tree in which no node can have more than two children.
- ▶ Every non-root node has a unique parent.
- ▶ Each element of a binary tree is called a node of the tree.
- ▶ Each node have Left and right child either null or pointing to someone.



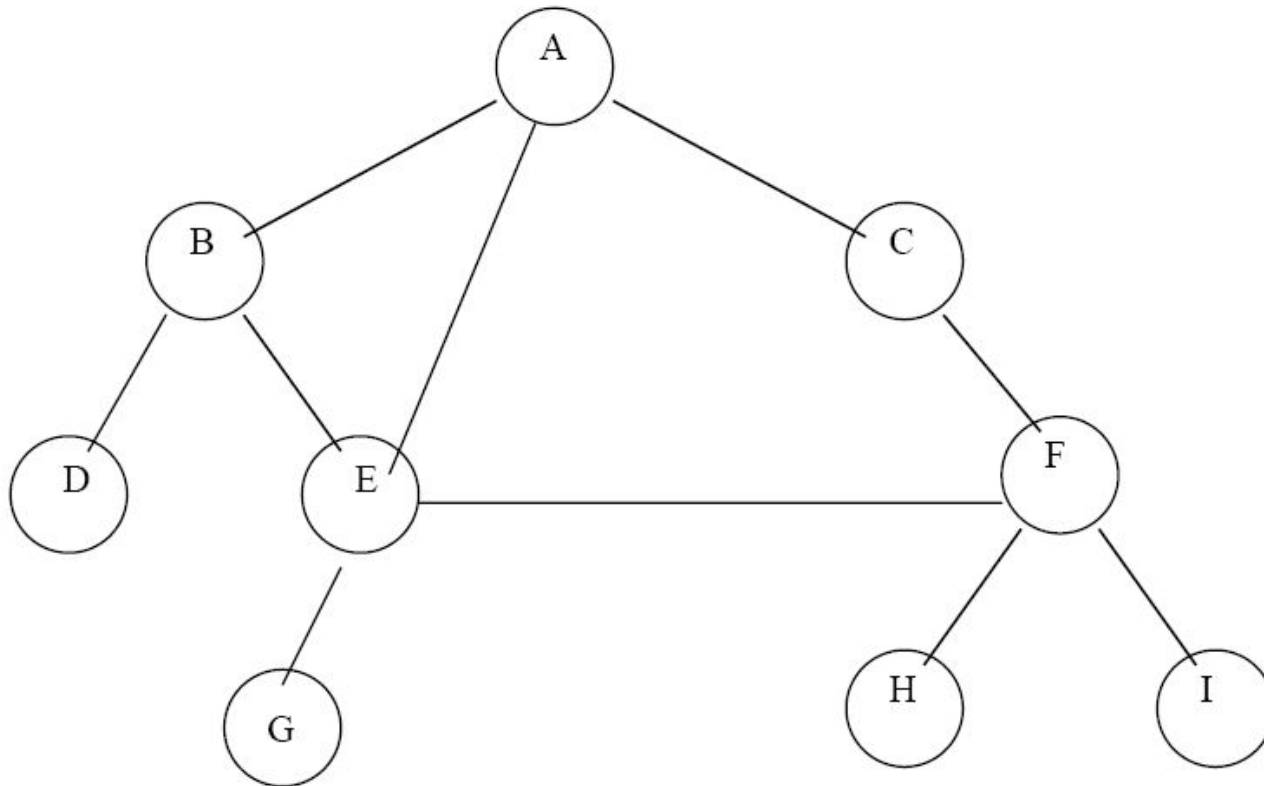
# Non-Tree



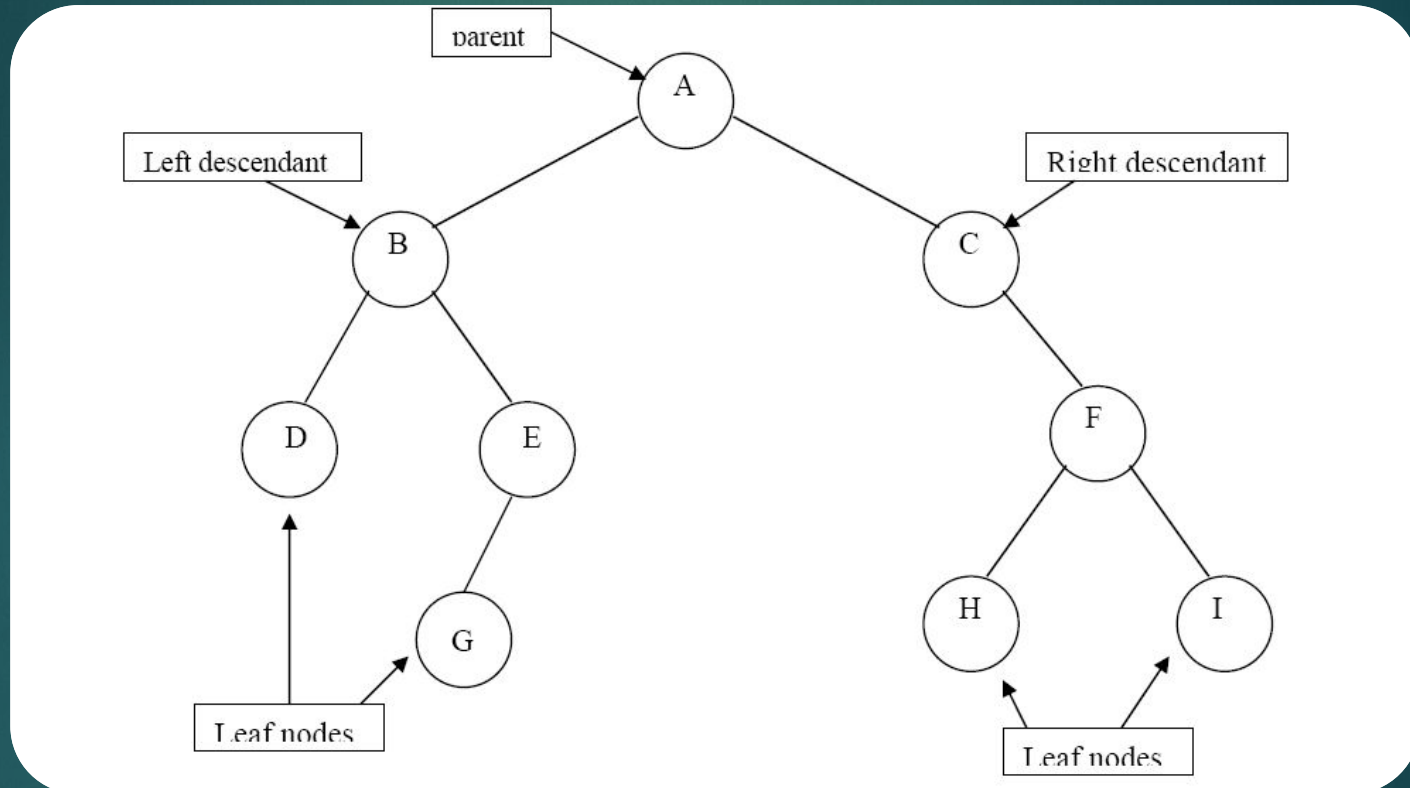
# Non-Tree



# Non-Tree



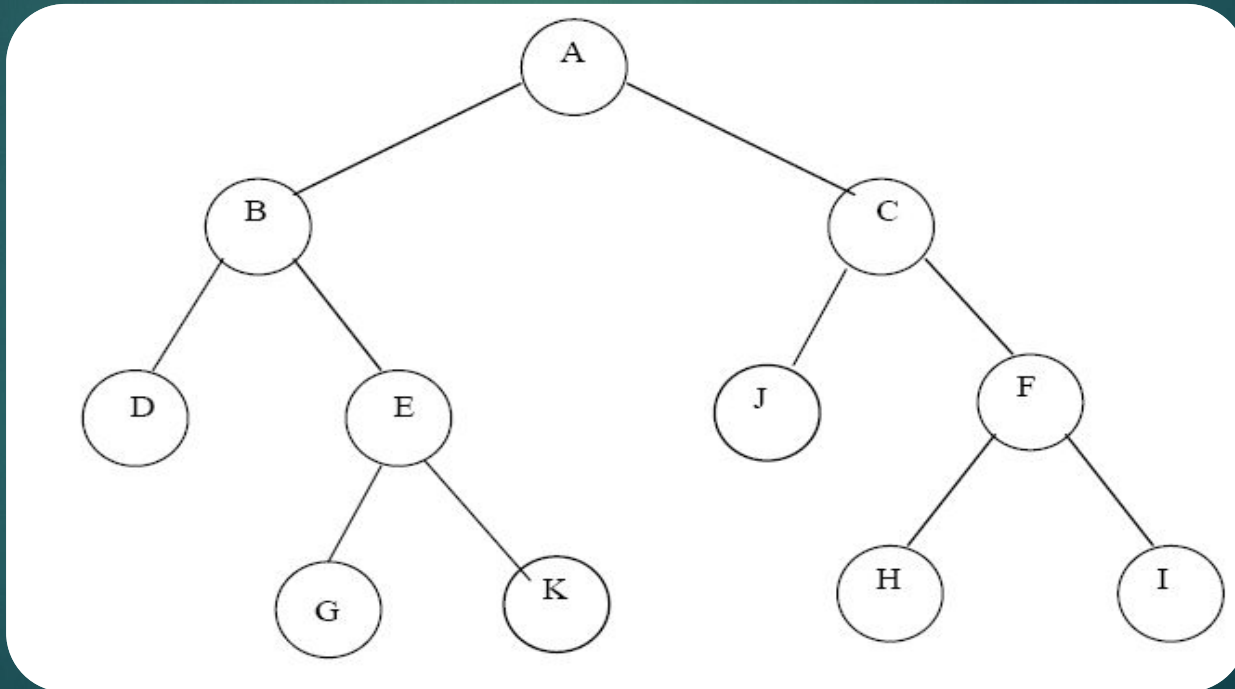
# Tree





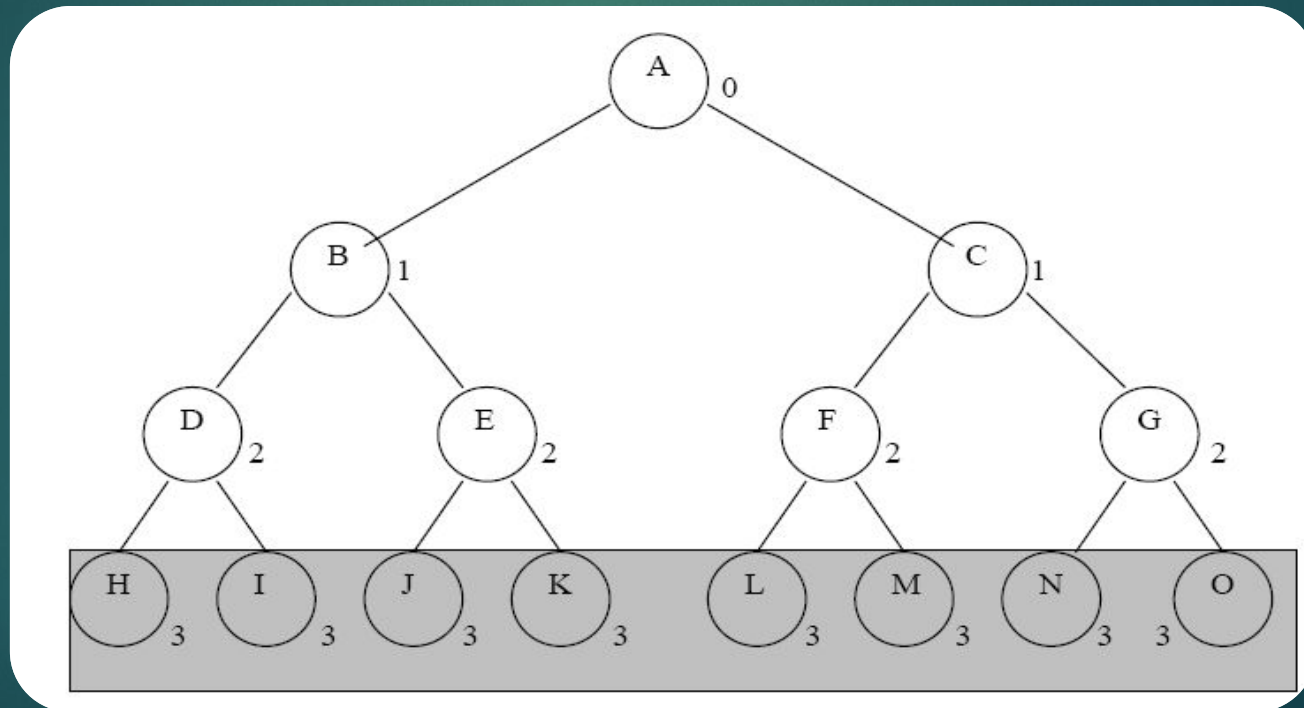
# Strictly Binary tree

- ▶ If every non-leaf node in a binary tree has non-empty left and right subtrees, the tree is termed as strictly binary tree.



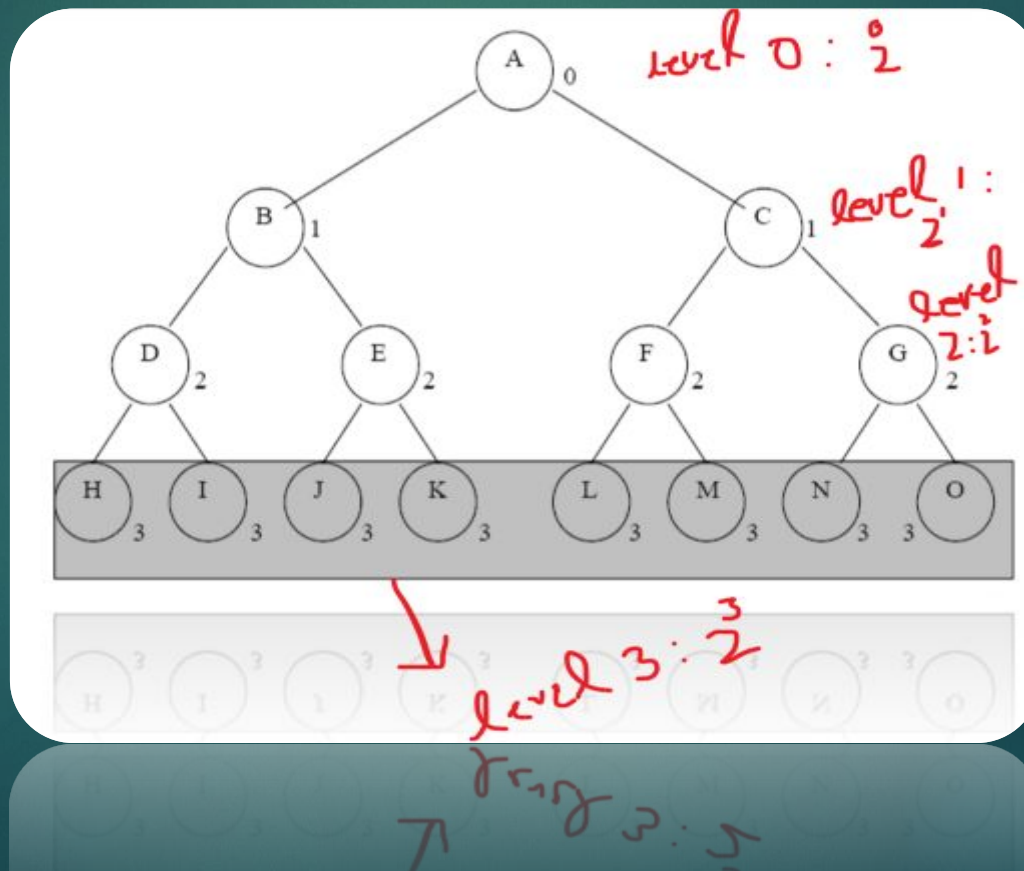
# Perfect Complete Binary Tree

- ▶ A complete binary tree of depth  $d$  is the strictly binary tree all of whose leaves are at level  $d$



# Number of nodes at each level in a complete binary tree

- At level  $k$ , there are  $2^k$  nodes.



❏ At level  $k$ , there are  $2^k$  nodes

❏ Total number of nodes in the tree of depth  $d$ :

$$\boxed{2^0 + 2^1 + 2^2 + \dots + 2^d = \sum_{j=0}^d (2^j) = 2^{d+1} - 1}$$

❏ In a complete binary tree, there are  $2^d$  leaf nodes and  $(2^d - 1)$  non-leaf (inner) nodes

$$n = 2^{d+1} - 1$$

$$\text{Or } \log_2(n+1) = d+1$$

$$\text{Or } d = \log_2(n+1) - 1$$

❏ For example, for  $n=15$ ,  $\log_2(15+1)$  is 4 so  $(4-1)=3$ ; the tree would be 3 levels deep.





If there are 'n' nodes:

- ❑ Maximum height of the binary tree= $n-1$
- ❑ Minimum height of the binary tree= $\text{floor}(\log_2 n)$

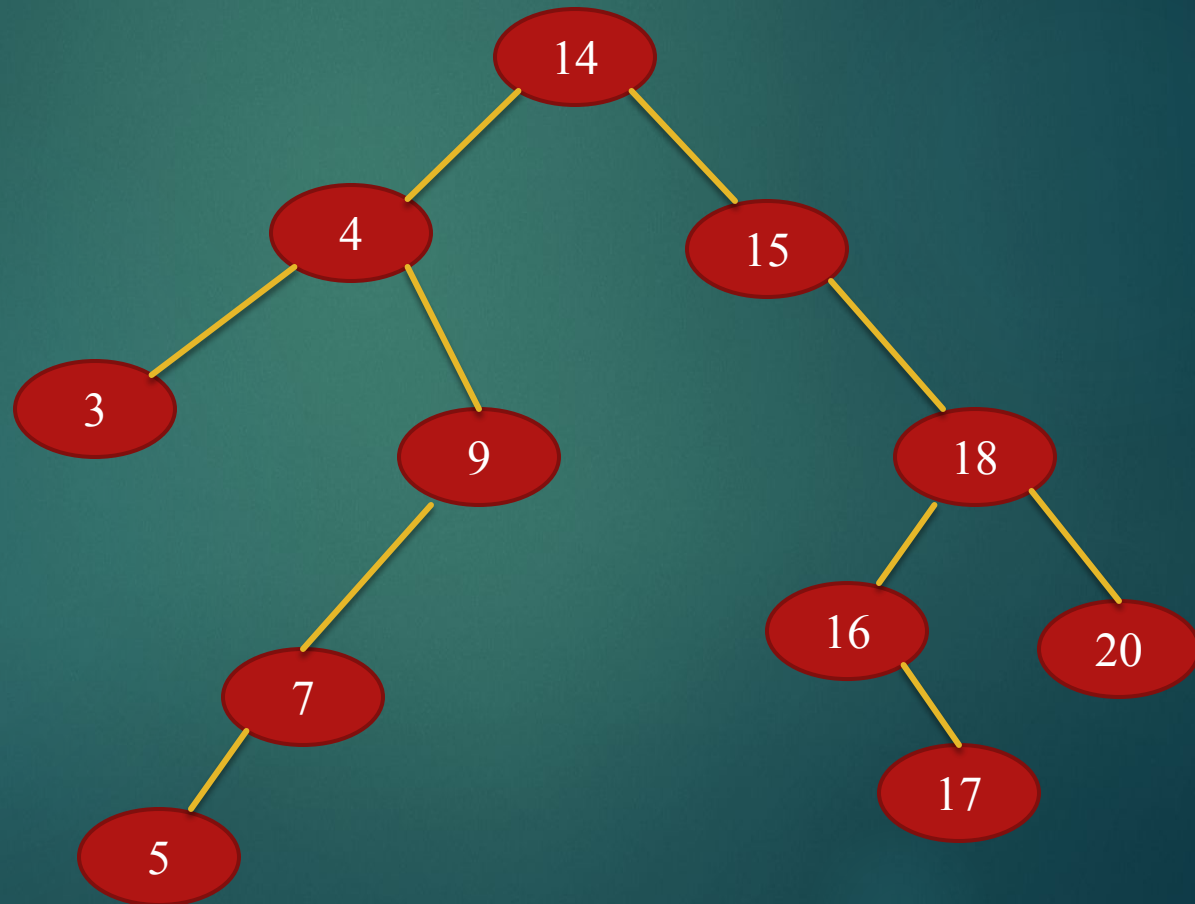
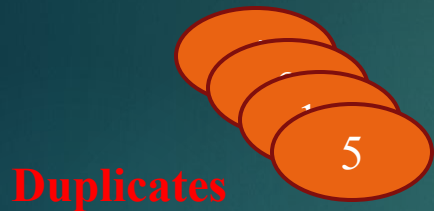
# Applications of Binary Tree

- ❑ **Searching for Duplicates**

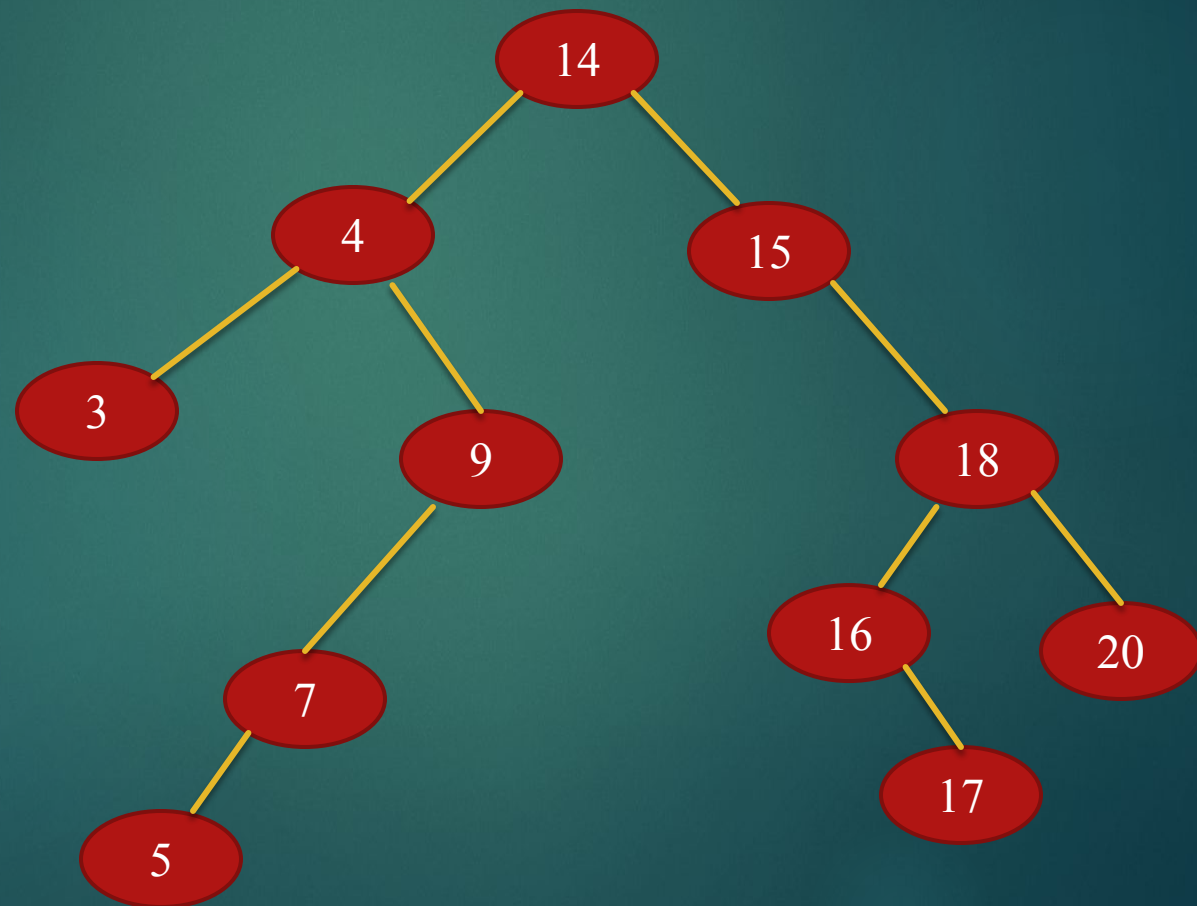
- ❑ For example, roll numbers, telephone numbers or voter's list.
- ❑ This list can be of very huge size ranging to thousands or millions.
- ❑ if the list of numbers is large and is growing.
- ❑ A linked list can handle the growth, but comparisons would still be large
- ❑ *Sample data*
  - ❑ 14, 15, 4, 9, 7, 18, 3, 5, 16, 4, 20, 17, 9, 14, 5

# Binary Tree

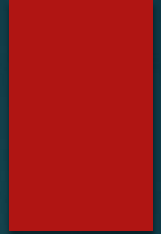
□ 14, 15, 4, 9, 7, 18, 3, 5, 16, 4, 20, 17, 9, 14, 5



- ▶ Preorder: 14,4,3,9,7,5,15,18,16,17,20
- ▶ Inorder: 3,4,5,7,9,14,15,16,17,18,20
- ▶ PostOrder: 3,5,7,9,4,17,16,20,18,15,14



# Deleting a node from BST

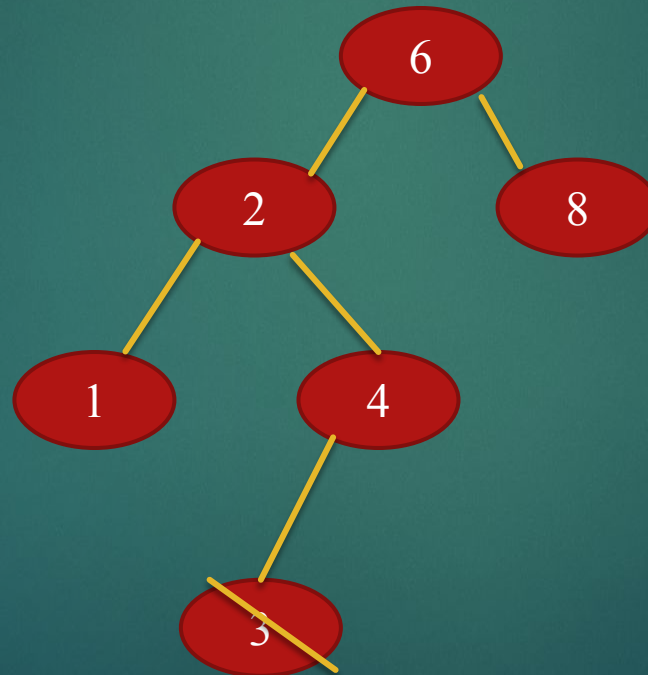


- ❑ **Case I:** If the node is a leaf, it can be deleted quite easily.
- ❑ **Case II:** The node to be deleted has either left child (subtree) or right child (subtree).
- ❑ **Case III:** The node to be deleted has both the left and right children (subtree).



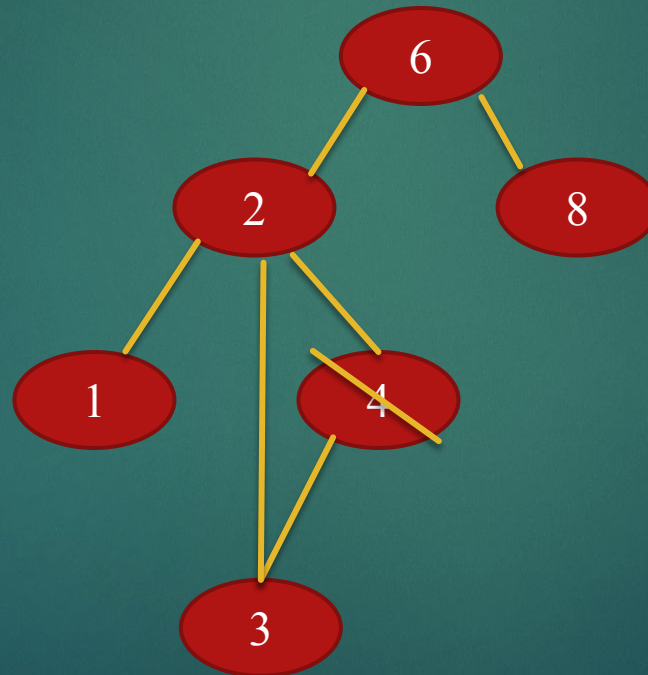
# Case-I:

- ▶ The node to be deleted is the leaf node i.e. it has no right or left child. It is very simple to delete such node.



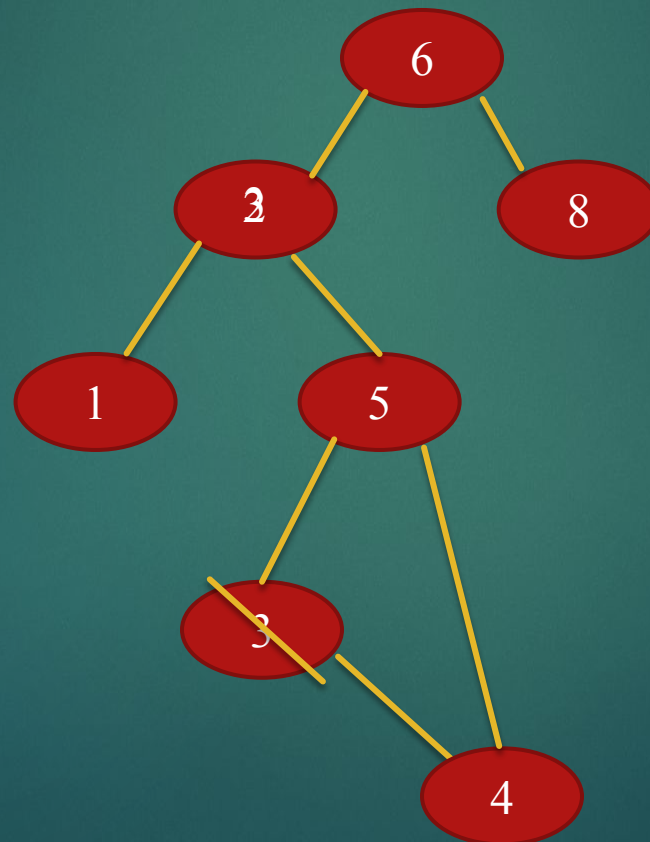
# Case-II:

- If the node has one child, the node can be deleted after its parent adjusts a pointer to bypass the node and connect to inorder successor.



# Case-III:

- When the node to be deleted has both left and right subtrees. The strategy is to replace the data of this node containing the smallest data of the right subtree and recursively delete that node.



- I. After we have found the left most node in the right subtree of the node containing number 2, we copy the contents of this left most node i.e. 3 to the node to be deleted with number 2.
- II. Next step is to delete the left most node containing value 3. Now being the left most node, there will be no left subtree of it, it might have a right subtree. Therefore, the deletion of this node is the case 2 deletion and the delete operation can be called recursively to delete the node.